

Toward Ever-Evolution Network Threats: A Hierarchical Federated Class-Incremental Learning Approach for Network Intrusion Detection in IIoT

Jingxin Mao¹, Zhiwei Wei¹, Graduate Student Member, IEEE, Bing Li², Member, IEEE,
Rongqing Zhang³, Member, IEEE, and Lingyang Song⁴, Fellow, IEEE

Abstract—The rise of collaborative manufacturing, driven by the rapid proliferation of Industrial Internet of Things (IIoT) technologies, has markedly enhanced agility and productivity in industrial environments. However, this advancement has also significantly broadened the attack surface and uncovered unique vulnerabilities intrinsic to these interconnected systems. This article introduces a novel hierarchical federated incremental learning network intrusion detection (HFIN) approach. To our knowledge, this is the first attempt to address the ever-evolution network intrusion detection (NID) challenges in IIoT landscapes from the continuous attack-defense perspective. Our proposed HFIN capitalizes on decentralized model training across multifarious IIoT devices, ensuring data privacy and empowering continuous learning capabilities. It utilizes distributed data sources for secure experience sharing, collaboratively enhancing the continuous detection performance of the global model. Furthermore, regarding the inherent resource constraints of IIoT devices, we proposed a novel edge-client weighted transmission optimization strategy (WTO). This strategy adeptly balances effective intrusion detection with the operational constraints of IIoT devices. By holistically considering detection capabilities and data volume across different attack types, it prioritizes the transmission of more critical and scarce attack data for training

within bandwidth constraints. This maintains the comprehensive detection capability of the global model against various network attacks. To validate the effectiveness of HFIN, we conduct extensive experiments using the NF-UQ-NIDS-v2 and NF-ToN-IIoT-v2 data sets. Experimental results demonstrate that our method outperforms baselines by approximately 10% in terms of accuracy and F1-score, highlighting the applicability and effectiveness of HFIN in enhancing security against sophisticated industrial environments and ever-evolving cyber threats.

Index Terms—Class-incremental learning (CIL), federated learning (FL), Industrial Internet of Things (IIoT), network intrusion detection (NID).

I. INTRODUCTION

THE INTERNET and Web expansion into the physical reality has enhanced the fourth industrial revolution, catalyzing the emergence of the network collaborative manufacturing paradigm. By fusing the physical components through interlinked digital networks, collaborative manufacturing connects elements, such as people, machines, workshops, factories, supply chains, and products [1]. Hereby, it establishes close relationships among enterprises, facilitates real-time data sharing, and enables coordinated decision-making across geographically dispersed manufacturing units for improving productivity, efficiency, and reliability [2].

While the collaborative manufacturing paradigm has significantly enhanced agility and productivity, its inherent interconnectedness has proportionally expanded the attack surface, exposing vulnerabilities in ways previously unseen in traditional manufacturing environments [3]. For instance, in standard network settings, attackers often gain control through compromised Industrial Internet of Things (IIoT) devices, leveraging the threat magnified in isolated systems but typically contained within their limited scope [4]. However, the collaborative manufacturing landscape presents a stark contrast. Due to the deeply interconnected nature of systems, a breach in one segment can have cascading effects, jeopardizing the entire supply chain. This interconnectedness, especially with the integration of information technology/operational technology (IT/OT) domain networks and cloud-based collaborative management, necessitates a reevaluation of traditional security concepts and strategies [1].

Manuscript received 24 April 2024; accepted 26 May 2024. Date of publication 3 June 2024; date of current version 6 September 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB3104200; in part by the National Natural Science Foundation of China under Grant 62271351 and Grant 62201390; in part by the Natural Science Foundation of Shanghai under Grant 22ZR1463600; in part by the Chenguang Program of Shanghai Education Development Foundation and Shanghai Municipal Education Commission under Grant 21CGA24; in part by the Hangzhou Qianjiang Special Expert and the Science and Technology Innovation Program of Hunan Province under Grant 2022RC4024; and in part by the Fundamental Research Funds for the Central Universities. Jingxin Mao and Zhiwei Wei contributed equally to this work. (Corresponding author: Rongqing Zhang.)

Jingxin Mao and Bing Li are with the School of Software Engineering, Tongji University, Shanghai 200092, China (e-mail: mjjx@tongji.edu.cn; lizi@tongji.edu.cn).

Zhiwei Wei is with the Shanghai Research Institute for Intelligent Autonomous Systems, Tongji University, Shanghai 201210, China (e-mail: 2311769@tongji.edu.cn).

Rongqing Zhang is with the School of Software Engineering, Tongji University, Shanghai 200092, China, and also with the Shanghai Research Institute for Intelligent Autonomous Systems, Tongji University, Shanghai 201210, China (e-mail: rongqingz@tongji.edu.cn).

Lingyang Song is with the National Engineering Laboratory for Big Data Analysis and Applications, School of Electronics, Peking University, Beijing 100871, China (e-mail: lingyang.song@pku.edu.cn).

Digital Object Identifier 10.1109/IJOT.2024.3408634

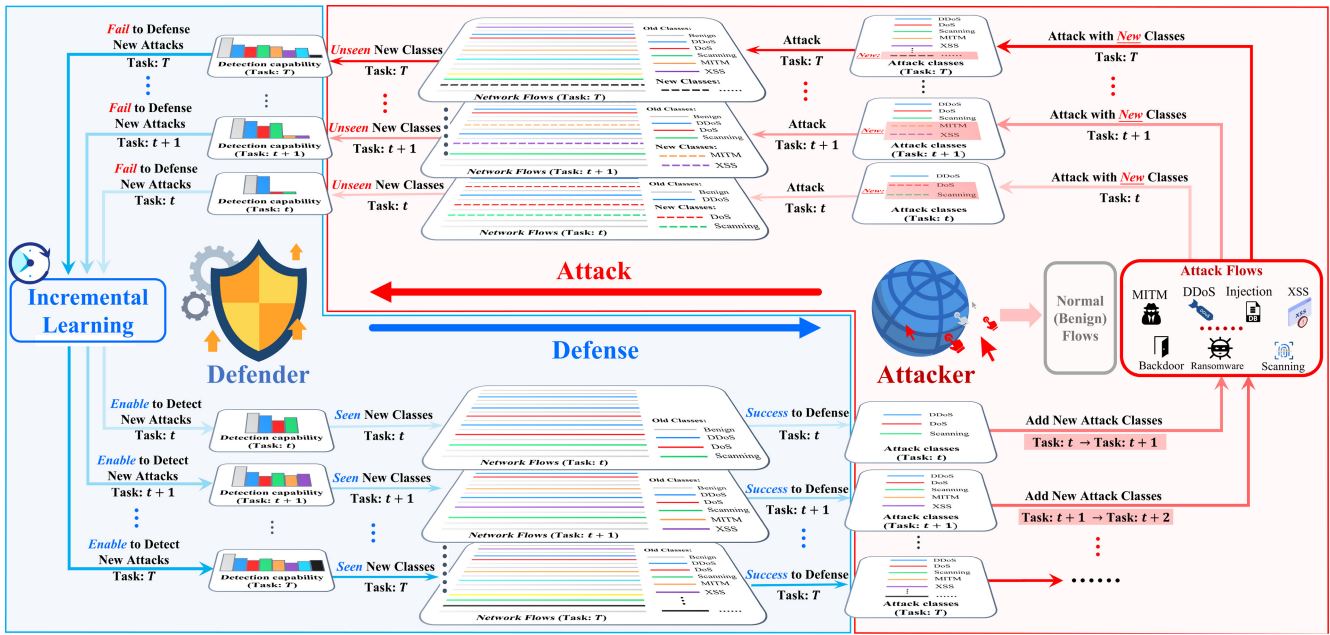


Fig. 1. Illustrate the dynamic and ever-evolution attack-defense scenario. Each evolution of attack and defense begins with the attackers launching new types of attacks. In the initial stages of the attack, the defenders have not encountered these novel attack types, rendering them unable to detect and defend against these attacks. To mount an effective defense, the defenders continuously engage in incremental learning, enabling them to develop detection capabilities against the new attacks. This enables defenders to defend against these new attacks, leading to a decrease in the success rate of the attack from attackers. Consequently, this prompts attackers to launch more attacks containing new types of threats, thus initiating the next cycle of attack-defense evolution.

To alleviate the security concerns in collaborative manufacturing environments, network intrusion detection (NID) has become a critical component [5]. NID is a security mechanism designed to monitor and analyze network traffic to identify signs of malicious attacks or violations of security policies. Delving into the collaborative manufacturing environment, a well-designed NID solution needs to address three critical challenges: First, it is essential to effectively safeguard IoT devices, which are not only primary targets in enterprise attacks but also often subject to limited computational resources, making them vulnerable to attacks [6]. Second, the collaborative nature of modern manufacturing, with its diverse and interdependent networks, calls for a security strategy that is inherently cooperative and experience-based. This is primarily attributed to attacks occurring upstream in the supply chain, potentially giving rise to a sequence of analogous attacks downstream, which is called the ripple effect [7]. Last but not least, the dynamic nature of collaborative manufacturing means that the landscape of potential cyber threats is continually evolving. As depicted in Fig. 1, the interconnected systems and processes in these environments collectively shape a continually evolving dynamic landscape of attack and defense. Attackers launch more diverse and novel attacks to enhance the success rate of attack, and defenders, in response, lean toward continuous learning to enhance their detection capabilities. Therefore, it is critical to develop a dynamic and adaptive security strategy to address the ever-evolving threat landscape.

In such context, traditional NID methods, such as signature-based and rule-based ones [8], [9], are becoming increasingly inadequate and less scalable in collaborative manufacturing environments. Although a series of deep learning-based

NID [10], [11], [12] solutions have been proposed recently to adapt to the dynamic and complex network environment, these methods have not delved into the possibilities of massive and heterogeneous device collaboration and ever-changing defending. In this article, we focus on the NID problem in IIoT environments and propose a novel hierarchical federated class-incremental learning-based NID (HFIL) approach. By integrating federated learning (FL) and class-incremental learning (CIL) techniques, the proposed HFIL approach allows for decentralized model training across diverse IIoT devices, ensuring data privacy and empowering continuous learning capabilities. It utilizes distributed data sources for secure experience sharing, collaboratively enhancing the continuous detection performance of the global model. Additionally, the CIL aspect of our method enables it to adaptively update and refine its detection capabilities in response to new and ever-evolving cyber threats, a critical feature in the fast-paced IIoT environment. The contributions of this article are summarized as follows.

- 1) We propose a hierarchical federated CIL approach for NID from a continuously attack-defense perspective. The proposed approach gathers heterogeneously distributed security-relevant data from IIoT devices, leverages the distributed computing resources of edge servers to train a global model while preserving data privacy, and employs CIL to adaptively update the model to address the dynamic and evolving threat landscape. To the best of our knowledge, this is the first proposal of a hierarchical federated CIL approach specifically designed for NID tasks in cloud-edge-client collaborative IIoT.
- 2) To further acknowledge the resource constraints of IIoT devices, our HFIL incorporates an edge-client WTO

strategy to balance the demand for effective intrusion detection with the operational limitations of IIoT devices. The proposed WTO strategy strives to strike an equilibrium between the exigency for effective intrusion detection and the operational limitations inherent in IIoT devices. Consequently, the system achieves dynamic modulation of the transmission rate from IIoT devices to edge servers, ensuring judicious resource utilization while upholding the comprehensive detection capability of the global model against various network attacks.

- 3) We construct incremental learning benchmarks for the NF-UQ-NIDS-v2 and NF-ToN-IoT-v2 data sets and conduct extensive experiments on these two data sets to evaluate the performance of the proposed HFIN method. The results demonstrate that our proposed approach outperforms baselines by approximately 10% in terms of accuracy and F1-score, validating the effectiveness and applicability of the proposed HFIN.

The remainder of this article is organized as follows. Section II reviews the related works, Section III introduces the system model and problem formulation, Section IV presents the proposed HFIN approach, Section V describes the evaluation metrics, Section VI presents the experimental results, and Section VII concludes this article.

II. RELATED WORKS

In this section, we review the related works on NID, FL-based NID, and CIL-based NID.

A. Network Intrusion Detection

NID is a security mechanism designed to monitor and analyze network traffic to identify signs of malicious attacks or violations of security policies. Recently, many deep learning-based solutions have been proposed to detect the dynamic and complex network intrusion. Lin et al. [13] presented an intrusion detection system featuring stacked sparse autoencoders and RNNs. He et al. [14] introduced an intrusion detection model leveraging multimodal deep autoencoders and LSTM to effectively integrate features from different levels of network connections. Hu et al. [15] proposed a deep learning-based one-class intrusion detection scheme to enhance the security of industrial networks. Andresini et al. [16] introduced a method that represents network flows as images and obtains their imagery representation through nearest neighbor search and clustering processes. While these methods have advanced research in NID to a certain extent, they face challenges in effectively applying them to complex IIoT scenarios, particularly in meeting data privacy requirements and addressing dynamic and continuous defense needs.

B. Federated Learning on NID

In the collaborative manufacturing paradigm, a myriad of devices are exposed to diverse network attacks, with the data then transmitted to the edge server within the local area for processing. However, edge servers typically possess limited computational and storage resources. This often necessitates

TABLE I
NOTATIONS AND DEFINITIONS

Notations	Definitions
$\mathcal{O}$	Cloud server
$\mathcal{E}_1, \dots, \mathcal{E}_n$	n edge servers, also denoted as $\mathcal{E}$
$\mathcal{C}_{i,1}, \dots, \mathcal{C}_{i,m}$	m client devices interacting with $\mathcal{E}_i$, also denoted as $\mathcal{C}_i$
$\mathcal{T}_i^t$	The data collected by $\mathcal{C}_i$ at the t -th incremental task
$\mathcal{K}^t$	The number of new classes for the t -th incremental task
$\mathcal{K}^{t,old}$	The number of old classes for the t -th incremental task
$\mathcal{D}_i^t$	The local training dataset formed by the transmission from $\mathcal{C}_i$ to $\mathcal{E}_i$ at the t -th incremental task
$\mathbf{x}_{i,j}^t, \mathbf{y}_{i,j}^t$	The j -th data pair comprising network flow data and corresponding class label in $\mathcal{D}_i^t$
$\mathcal{D}_i^{t,old}$	The sub-dataset of the $\mathcal{K}^{t,old}$ old classes of $\mathcal{D}_i^t$
N_i^t	The size of $\mathcal{D}_i^t$
$\mathcal{Y}^t$	The label space for the t -th incremental task
$\mathcal{M}_i$	Exemplar memory of $\mathcal{E}_i$
$\Theta_{\mathcal{E}_i}^{t,r}, \Theta_{\mathcal{O}}^{t,r}$	The dynamic defense models at the r -th communication round of the t -th incremental task for $\mathcal{E}_i$ and $\mathcal{O}$
$\Theta_{\mathcal{E}_i}^t, \Theta_{\mathcal{O}}^t$	The best dynamic defense models for $\mathcal{E}_i$ and $\mathcal{O}$ in the t -th incremental task

the further upload of data to the more powerful cloud for centralized training [17]. FL [18], emerging as a novel paradigm in distributed learning, enables collaborative model training among edge servers while safeguarding privacy data. This approach alleviates the burden on the central cloud server and enhances the overall system efficiency by facilitating decentralized training at the edge [19]. Mothukuri et al. [20] introduced an FL approach for intrusion detection in IoT networks. Liu et al. [21] combined FL with blockchain technology to develop an intrusion detection system for securing vehicular networks. Sun et al. [22] proposed a hierarchical FL intrusion detection system with transformer-based local models. Ruzafa-Alcázar et al. [23] evaluated the use of various differential privacy methods in FL-based NID. While existing FL-based NID methods have made significant strides in enhancing network security, they lack the flexible adaptability to continuously evolving network attacks within more complex IIoT environments. Specifically, there is a deficiency in dynamic security defense strategies to adapt to the continually evolving threat landscape characterized by the emergence of novel and diverse network attacks.

C. Class-Incremental Learning on NID

In the context of IIoT environments, a constantly evolving threat landscape characterized by the emergence of new and sophisticated attack techniques necessitates the adoption of dynamic security strategies to adapt to the ever-changing environment. CIL [24], [25], [26], [27] allows intrusion detection systems to adjust and learn when new threat classes arise, ensuring the model remains relevant over time. Currently, related works on CIL-based NID are as follows: ID-FSCIL [29] utilized meta-learning to extract new intrusion patterns from a small number of samples, thereby maximizing the model's generalization ability to handle emerging attacks. Amalapuram et al. [30] evaluated the applicability of continual learning in addressing challenges in the design of NID systems. Cerasuolo et al. [31] investigated how CIL methods

enhance the performance of intrusion detection systems in IoT networks. BFS-NID [32] employed a vision transformer (ViT) [33] to fixate the learning of feature representations, preventing catastrophic forgetting. Additionally, it employed a branch fusion strategy to associate the context of learned attack classes with new classes in different sessions. While these approaches explored or evaluated the application of CIL in NID, there is a lack of consideration for data privacy security.

III. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we first propose a hierarchical federated CIL approach for NID. Then, we formulate the optimization problem both in terms of the dynamic defense learning model and the data transmission model. The notations are listed in Table I.

A. System Overview

The incorporation of FL in industrial NID is strategically designed to mitigate sophisticated cyber threats and unauthorized access attempts in interconnected industrial networks. To fully leverage the benefits of FL, utilize the distributed computing resources of edge servers, and address the challenges of dynamic and persistent attack-defense scenarios, a hierarchical federated CIL framework is proposed with the following critical components.

- 1) *Cloud Server*: The cloud server is used to manage and disseminate global security policies. In FL, it is responsible for aggregating the locally updated models from edge servers to train the global model without privacy leakage.
- 2) *Edge Server*: The edge servers are intermediary processing layers between client devices and the cloud, collecting and processing network flows from client devices. In the industrial FL process, client devices have limited computing resources and cannot train the model locally. Thus, the edge servers are responsible for training local models and uploading them to the cloud server.
- 3) *Client Device*: Each edge server $\mathcal{E}_i$ has a local data set $\mathcal{D}_i$ collected from M client devices within their scopes. The client devices serve as the primary entry points for cyber-attacks, and present numerous attack surfaces due to their widespread deployment and direct interaction with the physical environment. They are responsible for collecting network flows and transmitting security-relevant data to the edge servers for further analysis.

The workflow starts with the client devices collecting network flows and transmitting them to the edge servers. Subsequently, the edge servers convey the dynamic defense model to the cloud server for aggregation, enhancing dynamic defense capabilities while preserving data privacy. In the context of the hierarchical cloud-edge-client architecture, client devices play a pivotal role in transmitting extensive data to the edge server. This transmission serves as the fundamental basis for dynamic defense learning at the edge server. Concurrently, collaborative learning between the cloud server and edge servers is instituted to bolster dynamic defense capabilities, all

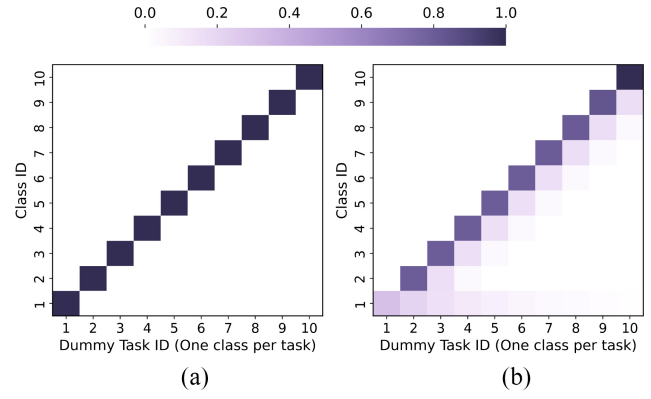


Fig. 2. Illustrate the distribution probability of label space: (a) conventional CIL and (b) HFIN. Following convention, the initial class ID (set here as 1) denotes the normal class (Benign).

while ensuring the paramount consideration of safeguarding data privacy. Therefore, it is important to clarify two system models, pointing out what to transmit between edge servers and the cloud server (dynamic defense learning model) and how to transmit between client devices and the edge server (data transmission model).

B. Dynamic Defense Learning Model

In contrast to nonincremental learning where the number of classes remains constant all the time, CIL involves a sequence of streaming tasks $\mathcal{T} = \{\mathcal{T}^t\}_{t=1}^T$, where T signifies the total number of incremental tasks. The t th incremental task $\mathcal{T}^t = \{\mathbf{x}_j^t, \mathbf{y}_j^t\}_{j=1}^{N^t}$ consists of N^t pairs of samples $\mathbf{x}_j^t$ and their corresponding labels $\mathbf{y}_j^t \in \mathcal{Y}^t$. Here, $\mathbf{x}_j^t \in \mathbb{R}^d$ denotes input network flows with d features, and $\mathcal{Y}^t$ represents the label space of the t th incremental task.

In the traditional CIL, the label space $\mathcal{Y}^t$ comprises only $\mathcal{K}^t$ new classes that are different from $\mathcal{K}^{t,\text{old}} = \sum_{k=1}^{t-1} \mathcal{K}^k \subset \cup_{l=1}^{t-1} \mathcal{Y}^l$ old classes in previous $t-1$ tasks, as illustrated in Fig. 2(a). For each dummy task with only one class, it holds data solely for the current class. However, this assumption in traditional CIL is overly idealistic and lacks practical feasibility. In the IIoT paradigm, the label space $\mathcal{Y}^t$ within the t th task should not only encompass $\mathcal{K}^t$ new classes but also include $\mathcal{K}^{t,\text{old}}$ old classes. In fact, it is unrealistic to assume that all previous attack classes would cease their attacks in this incremental task. For example, normal (Benign) traffic flows exist across the entire CIL process, rather than solely existing in the initial 1th dummy task. Therefore, we model the label space distribution of $\mathcal{Y} = \cup_{t=1}^T \mathcal{Y}^t$ as follows:

$$\boldsymbol{\pi} = [\boldsymbol{\pi}_1, \boldsymbol{\pi}_2, \dots, \boldsymbol{\pi}_K]^T \in \mathbb{R}^{K \times K} \quad (1)$$

where K is the number of total classes and $\boldsymbol{\pi}_k \in \mathbb{R}^K$ ($k = 1, 2, \dots, K$) is the distribution of the k th class as follows:

$$\boldsymbol{\pi}_k = [\pi_{k,1}, \pi_{k,2}, \dots, \pi_{k,K}] \sim \text{Poly}(\boldsymbol{\pi}_k | \alpha_k) \quad (2)$$

$$\pi_{k,l} = \begin{cases} 0 & \text{for } 1 \leq l < k \\ \frac{\alpha_k}{1 - (1 - \alpha_k)^{K-l+1}} & \text{for } l = k \\ \frac{\alpha_k \cdot (1 - \alpha_k)^{l-k}}{1 - (1 - \alpha_k)^{K-l+1}} & \text{for } k < l \leq K \end{cases} \quad (3)$$

where $\alpha_k \in \boldsymbol{\alpha} \subset (0, 1]^K$ is the hyperparameter for the label space distribution of the k th class. For ease of representation,

we adopt $\text{Poly}(\alpha)$ to denote the label space distribution. Significantly, traditional CIL employs the label space distribution of $\text{Poly}(\mathbf{1}^K)$, whereas we adopt $\text{Poly}(\alpha)(\mathbf{0} < \alpha < \mathbf{1})$ as illustrated in Fig. 2(b). Moreover, we follow [25], [26], [27], [28] to construct an exemplar memory $\mathcal{M}_i$ for each edge $\mathcal{E}_i$. This memory is utilized to select and reserve $(|\mathcal{M}_i|/\mathcal{K}^{t,\text{old}})$ exemplars for each old class in the t th incremental task, where $(|\mathcal{M}_i|/\mathcal{K}^{t,\text{old}}) \ll (N_i^t/\mathcal{K}^t)$.

In the context of dynamic defense learning, for the r th global communication round ($r = 1, 2, \dots, R$), all edge servers $\mathcal{E}$ participate in the learning process. Specifically, each edge server $\mathcal{E}_i$ will receive the latest global model $\Theta_{\mathcal{O}}^{t,r}$, and train $\Theta_{\mathcal{O}}^{t,r}$ on $\mathcal{D}_i^t \cup \mathcal{M}_i \sim \mathcal{P}_i^{|\mathcal{D}_i^t|+|\mathcal{M}_i|}$, where $\mathcal{D}_i^t = \{\mathbf{x}_{i,j}^t, \mathbf{y}_{i,j}^t\}_{j=1}^{N_i^t} \subset \mathcal{T}_i^t$ is the data received by $\mathcal{E}_i$ at the t th incremental task, and $\mathcal{P}_i$ signifies the class distribution of the i th edge server. In the t th incremental task, the label space $\mathcal{Y}^t$ of each edge server encompasses $\mathcal{K}^t$ new classes alongside $\mathcal{K}^{t,\text{old}} = \sum_{k=1}^{t-1} \mathcal{K}^k \subset \cup_{l=1}^{t-1} \mathcal{Y}^l$ old classes. The data set of old classes for the t th incremental task is denoted as $\mathcal{D}_i^{t,\text{old}} = \{\mathbf{x}_{i,j}^t, \mathbf{y}_{i,j}^t\}_{j=1}^{N_i^t} \in \cup_{l=1}^{t-1} \mathcal{Y}^l \subset \mathcal{D}_i^t$, and it satisfies $|\mathcal{D}_i^{t,\text{old}}| \ll |\mathcal{D}_i^{t-1}|$ for all attack classes (i.e., class ID is more than 1). Upon loading $\Theta_{\mathcal{O}}^{t,r}$ and conducting the local training at the t th incremental task, $\mathcal{E}_i$ procures an updated local model $\Theta_{\mathcal{E}_i}^{t,r}$. All locally updated models of $\mathcal{E}$ are then uploaded to the cloud $\mathcal{O}$ to be aggregated as the global model $\Theta_{\mathcal{O}}^{t,r+1}$ for the next communication round. Subsequently, the cloud $\mathcal{O}$ distributes $\Theta_{\mathcal{O}}^{t,r+1}$ to all edge servers $\mathcal{E}$ for the next communication round.

C. Data Transmission Model

In the context of industrial NID, the client devices are typically dispersed across a wide geographical area, each occupying a unique position within the industrial environment. To accurately model data transmission, the location of client device $\mathcal{C}_{i,j}$ is represented by a 2-D coordinate (lx_i, ly_i) , where lx_i and ly_i correspond to its longitudinal and latitudinal positions, respectively. Then, the path loss model in the industrial environment is established by referring to [34]. Given transmitter $\mathcal{C}_{i,j}$ and receiver $\mathcal{E}_i$, the path loss between them is defined as

$$\text{PL}(\mathcal{C}_{i,j}, \mathcal{E}_i) = \text{PL}(f, d_0) + 10n_{pl} \log(d/d_0) + \text{SF} \quad (4)$$

where f is the carrier frequency, $\text{PL}(f, d_0)$ is the close-in free space PL in dB, d_0 is the reference distance, $d = \sqrt{(lx_{i,j} - lx_i)^2 + (ly_{i,j} - ly_i)^2}$ is the distance in the unit of meters, n_{pl} is the path loss factor, and SF is the shadow fading. In this article, we set $f = 3.5$, $d_0 = 1$, $n_{pl} = 2.5$ [34], and SF is a Gaussian random variable with zero mean and standard deviation of 8 dB. According to Shannon–Hartley theorem, the transmission rate between $\mathcal{C}_{i,j}$ and $\mathcal{E}_i$ is

$$R_{i,j} = B \log \left(1 + \frac{P_i^t \cdot 10^{\frac{-\text{PL}(\mathcal{C}_{i,j}, \mathcal{E}_i)}{10}}}{N_0} \right) \quad (5)$$

where B is the bandwidth, P_i^t is the transmission power, and N_0 is the noise power. Similar to [35], time division multiple

access (TDMA) is adopted to avoid interference between different client devices.

D. Problem Formulation

The objective of HFIN is to train a global model $\Theta_{\mathcal{O}}^T$ for the incremental tasks $\mathcal{T}$, subject to the transmission constraints of the client devices and the privacy constraints of the edge servers. The general optimization problem is formulated as follows:

$$\min_{\{\Theta, \{\delta_{i,j}[ts_{t,p}]\}\}} \sum_t \mathcal{L}^t(\Theta^t, \mathcal{D}^t) \quad (6a)$$

$$\text{s.t.} \quad |\mathcal{D}_i^t[ts_{t,p}]| \leq \delta_{i,j}[ts_{t,p}] R_{i,j}[ts_{t,p}] \quad \forall t, p, \mathcal{C}_{i,j}, \mathcal{E}_i \quad (6b)$$

$$\sum_i \delta_{i,j}[ts_{t,p}] \in \{0, 1\} \quad \forall t, p, \mathcal{C}_{i,j}, \mathcal{E}_i \quad (6c)$$

where $\mathcal{L}^t(\Theta^t, \mathcal{D}^t)$ is the loss function of the global model during the t th incremental task, $\mathcal{D}^t$ is the data set, $ts_{t,p}$ is the p th time slot for the t th incremental task, and $\delta_{i,j}[ts_{t,p}]$ is the binary variable indicating whether the device $\mathcal{C}_{i,j}$ transmits data to the i th edge server $\mathcal{E}_i$ in time slot $ts_{t,p}$.

However, direct optimization of this problem presents substantial challenges. The FL process in HFIN, particularly under the approach of CIL with heterogeneous data distributions, resembles a “black box.” The opaque relationship between the loss and the underlying data set, especially throughout the incremental learning stages, renders traditional optimization approaches less effective. This complexity is further compounded by the dynamic and evolving nature of cyber threats in industrial settings, where new attack classes emerge continually, altering the learning landscape unpredictably. To navigate these challenges, we delve into the features of incremental learning and propose a novel optimization approach to address the aforementioned issues in the next section.

IV. CLOUD-EDGE-CLIENT FEDERATED INCREMENTAL LEARNING ON INDUSTRIAL NETWORK INTRUSION

In this section, we introduce the proposed approach of HFIN. Fig. 3 and Algorithm 1 provide the overview and procedure of our proposed HFIN, respectively. We first introduce the details of the edge-client data transmission optimization, followed by the model structure design and cloud-edge federated CIL.

A. Edge-Client Weighted Transmission Optimization

Federated CIL is an iterative process, where the NID model performance may fluctuate due to the data set collected during each task $\mathcal{T}^t$. Addressing these fluctuations necessitates a dynamic approach, one that can adapt the data size for each attack type in accordance with the evolving landscape of network threats. Such adaptability ensures sustained relevance and efficacy of the model against contemporary intrusion patterns.

This article introduces an edge-client WTO, a methodology structured around three fundamental steps: 1) assumed detection F1-score evaluation; 2) attack type weight calculation; and

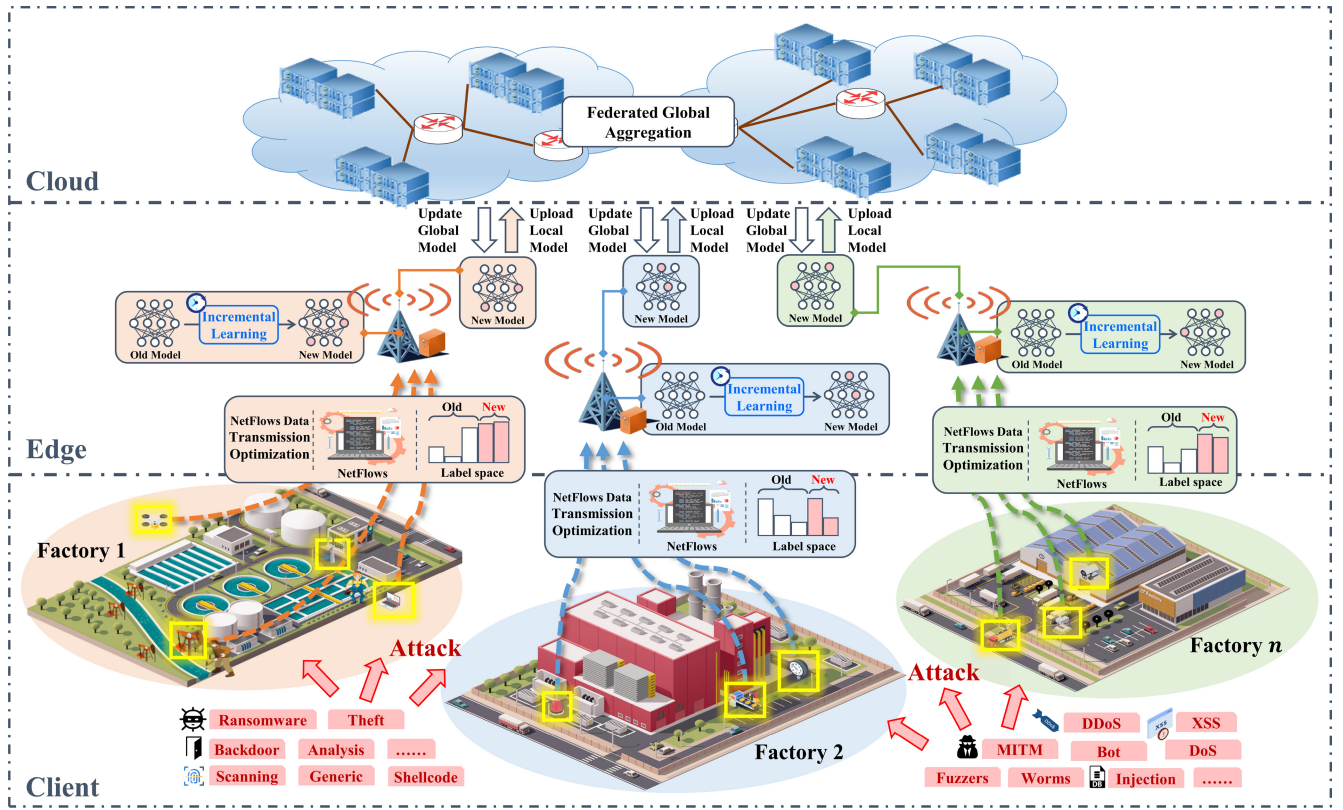


Fig. 3. Illustrate the framework of HFIN in IIoT.

3) data transmission optimization. During each task, the IIoT devices continuously detect the network intrusion activities. By using techniques, such as attack traceability methods, an evaluation of the actual detection F1-score can be obtained for the current global model. Then, the edge servers collect those performance metrics and upload them to the cloud, introducing an F1-score upper bound. This presupposed upper bound is used to facilitate subsequent comparisons to gauge improvements or regressions in model performance. After that, given the F1-score upper bound (provided by the cloud) and the regional actual F1-score (evaluated at the edge), the weight assigned to each attack type is computed by the following equation:

$$WTO^\beta : w_k^t = \frac{F1_k^*}{F1_k^t \cdot (\text{Num}_k^t)^\beta} \quad (7)$$

where $F1_k^*$ is the detection F1-score upper bound for the k th class, $F1_k^t$ is the actual detection F1-score of the model at the t th incremental task for the k th class, Num_k^t is the number of collection samples for the k th class before transmission, and β is a hyperparameter. The underlying principle of this formula is that a larger data set size or a precision exceeding the assumed level results in a proportionately lower weight, implying a diminished necessity for further data transmission of that specific type. Specifically, when $k \in \mathcal{Y}_i^t$ is a new type during the t th incremental task, which indicates that $F1_k^t$ is unknown, $F1_k^t = F1_k^*$ is set to ensure that the weight is not zero.

The foundational element of our approach is the bespoke weight assigned to each attack type for each client device.

As previously elucidated, this weight is computed based on the ratio of assumed accuracy to actual accuracy, adjusted by the collected sample size for each attack type. This weight inherently reflects the relative importance and urgency of transmitting specific types of data from the client devices to the centralized model. Therefore, the principal objective of our method is to maximize the weighted transmission, and the optimization problem is formulated as follows:

$$\max_{\{\delta_{i,j}[ts_{t,p}]\}} \sum_{t,p,i,j} w_k^t \cdot \delta_{i,j}[ts_{t,p}] \cdot R_{i,j}[ts_{t,p}] \quad (8a)$$

$$\text{s.t.} \quad |\mathcal{D}_i^t[ts_{t,p}]| \leq \delta_{i,j}[ts_{t,p}] R_{i,j}[ts_{t,p}] \quad \forall t, p, C_{i,j}, \mathcal{E}_i \quad (8b)$$

$$\sum_i \delta_{i,j}[ts_{t,p}] \in \{0, 1\} \quad \forall t, p, C_{i,j}, \mathcal{E}_i. \quad (8c)$$

This goal diverges from traditional approaches that aim to ensure equitable bandwidth distribution among all participating devices. Instead, our focus is on aggregating data that is most critical from a centralized perspective. To achieve this objective, we introduce a weighted transmission optimization approach, underpinned by the utilization of a priority queue. The priority is calculated as the product of the weight and the transmission rate, and the queue is sorted in descending order of priority. In each task, the edge server $\mathcal{E}_i$ selects the clients with the highest priority and manipulates the data transmission.

By adopting this WTO, we ensure that our HFIN approach is not merely collecting data uniformly across all nodes, but strategically focusing on data that is most impactful for the detection and prevention of network intrusions. This targeted approach is particularly beneficial in scenarios where

Algorithm 1: Hierarchical Federated Class Incremental Learning on Industrial NID

Input: number of tasks: T ; number of communication rounds R ; number of edge servers n ; evaluation interval e ;

Output: global model $\Theta_{\mathcal{O}}^T$

```

1 Cloud  $\mathcal{O}$  executes:
2 Initialize  $\Theta_{\mathcal{O}}^0$ 
3 for  $t = 1, 2, \dots, T$  do
4    $\Theta_{\mathcal{O}}^{t,1} \leftarrow \Theta_{\mathcal{O}}^{t-1}$ 
5    $\triangleright$  for DER:
6      $\Theta_{\mathcal{O}}^{t,1} \leftarrow \text{Dynamical Expansion}(\Theta_{\mathcal{O}}^{t,1})$ 
7   for  $i = 1, 2, \dots, n$  do
8     Clients  $\mathcal{C}_i$  collect network flows  $\mathcal{T}_i^t$ 
9      $\mathcal{D}_i^t \leftarrow \text{WTO}(\mathcal{C}_i, \mathcal{T}_i^t)$  via (8)
10  for  $r = 1, 2, \dots, R-1$  do
11    for  $i = 1, 2, \dots, n$  in parallel do
12      Send the global model  $\Theta_{\mathcal{O}}^{t,r}$  to  $\mathcal{E}_i$ 
13       $\Theta_{\mathcal{E}_i}^{t,r} \leftarrow \text{LocalTraining}(t, \mathcal{E}_i, \Theta_{\mathcal{O}}^{t,r})$ 
14       $\Theta_{\mathcal{O}}^{t,r+1} \leftarrow \sum_{i=1}^n \frac{|\mathcal{D}_i^t \cup \mathcal{M}_i| |\Theta_{\mathcal{E}_i}^{t,r}|}{\sum_{j=1}^n |\mathcal{D}_j^t \cup \mathcal{M}_j|}$ 
15     $\Theta_{\mathcal{O}}^t \leftarrow \text{Evaluate}\{\Theta_{\mathcal{O}}^{t,r} \mid r \bmod e = 0, 1 \leq r \leq R\}$  via (15)
16     $\{\mathcal{M}_i\}_{i=1}^n \leftarrow \text{Rehearsal}(\{\mathcal{M}_i\}_{i=1}^n, \mathcal{K}^{t,\text{old}}, \mathcal{K}^t)$ 
17     $\triangleright$  for WA:
18       $\Theta_{\mathcal{O}}^t \leftarrow \text{Weight aligning}(\Theta_{\mathcal{O}}^t)$ 
19 return  $\Theta_{\mathcal{O}}^T$ 

```

bandwidth is limited and where the significance of transmitted data varies markedly across different client devices and attack types.

B. Model Structure Design

For the precise classification of attack classes of NetFlow traffic, we employ a 1-D Convolutional Neural Network (1-D-CNN) as the feature extractor. Additionally, to address the continuously emerging attack classes in dynamic and persistent attack-defense scenarios, we integrate an adaptive fully connected layer as the classification head, composed of the previous task's old classification head and the current task's new classification head. The variable fully connected layer enables the model with incremental learning capabilities and facilitates continuous adaptation to new attack classes, collectively utilized to output classification predictions.

The designed network architecture primarily consists of 1-D convolutional layers, ReLU activation layers, 1-D batch normalization layers, max-pooling layers, and fully connected layers, as depicted in Fig. 4. More specifically, we employ four 1-D convolutional layers with a kernel size of 3, no padding, and a stride of 1, with output dimensions of 32, 32, 64, and 64, respectively. Each convolutional layer is subsequently followed by a ReLU activation layer and a 1-D batch normalization layer, collectively constituting a convolutional module.

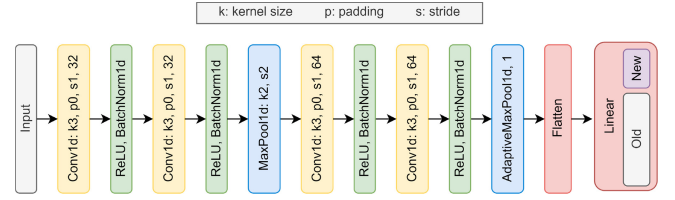


Fig. 4. Structure of designed 1-D-CNN for HFIN.

Additionally, following the initial two convolutional modules and the latter two, we apply max-pooling and adaptive max-pooling to downsample the convolutional results, respectively. Finally, a variable fully connected layer serves as the classification head, yielding classification logits output.

C. Cloud-Edge Federated Class-Incremental Learning

1) *Local Training:* At the t th incremental task, given the i th edge server $\mathcal{E}_i \in \mathcal{E}$ with the training data set $\mathcal{D}_i^t$ and exemplar memory $\mathcal{M}_i$, we employ the network architecture illustrated in Section IV-B to conduct training on $\Theta_{\mathcal{E}_i}^{t,r}$ by minimizing the classification loss $\mathcal{L}_{\text{CE}}$ for each minibatch $\{\mathcal{X}_{i,b}^t, \mathcal{Y}_{i,b}^t\} = \{\mathbf{x}_{i,j}^t, \mathbf{y}_{i,j}^t\}_{j=1}^b \subset \mathcal{D}_i^t \cup \mathcal{M}_i$ as follows:

$$\mathcal{L}_{\text{CE}} = \frac{1}{b} \sum_{j=1}^b \mathcal{H}(\mathcal{F}(\mathbf{x}_{i,j}^t; \Theta_{\mathcal{E}_i}^{t,r}), \mathbf{y}_{i,j}^t) \quad (9)$$

where b is the batch size, $\mathcal{H}(\cdot, \cdot)$ refers to the cross-entropy loss, $\Theta_{\mathcal{E}_i}^{t,r}$ is the NID model of the i th edge server $\mathcal{E}_i$, transmitted from the cloud $\mathcal{O}$ at the r th communication round for the t th incremental task. $\mathcal{F}(\mathbf{x}_{i,j}^t; \Theta_{\mathcal{E}_i}^{t,r}) \in \mathbb{R}^{\mathcal{K}^{t,\text{old}} + \mathcal{K}^t}$ represents the Softmax output of given input $\mathbf{x}_{i,j}^t$ via $\Theta_{\mathcal{E}_i}^{t,r}$.

During the training of the local model $\Theta_{\mathcal{E}_i}^{t,r}$, the Softmax output (i.e., probability predicted) by $\Theta_{\mathcal{E}_i}^{t,r}$ signifies interclass semantic similarity relations. To ensure semantic consistency among old classes across different incremental tasks and thus alleviate catastrophic forgetting as much as possible, we also conduct training on $\Theta_{\mathcal{E}_i}^{t,r}$ by minimizing the distillation loss $\mathcal{L}_{\text{KD}}$ for each old minibatch $\{\mathcal{X}_{i,b}^{t,\text{old}}, \mathcal{Y}_{i,b}^{t,\text{old}}\} = \{\mathbf{x}_{i,j}^{t,\text{old}}, \mathbf{y}_{i,j}^{t,\text{old}}\}_{j=1}^b \subset \mathcal{D}_i^{t,\text{old}} \cup \mathcal{M}_i$ as follows:

$$\mathcal{L}_{\text{KD}} = -\frac{1}{b} \sum_{j=1}^b \hat{\mathbf{p}}_{i,j}^{t,\text{old}} \log \mathbf{p}_{i,j}^{t,\text{old}} \quad (10)$$

$$\hat{\mathbf{p}}_{i,j}^{t,\text{old}} = \text{sg} \left(\frac{e^{f(\mathbf{x}_{i,j}^{t,\text{old}}; \Theta_{\mathcal{O}}^{t-1})/\tau}}{\sum e^{f(\mathbf{x}_{i,j}^{t,\text{old}}; \Theta_{\mathcal{O}}^{t-1})/\tau}} \right) \quad (11)$$

$$\mathbf{p}_{i,j}^{t,\text{old}} = \frac{e^{f_{\mathcal{K}^{t,\text{old}}}(\mathbf{x}_{i,j}^{t,\text{old}}; \Theta_{\mathcal{E}_i}^{t,r})/\tau}}{\sum e^{f_{\mathcal{K}^{t,\text{old}}}(\mathbf{x}_{i,j}^{t,\text{old}}; \Theta_{\mathcal{E}_i}^{t,r})/\tau}} \quad (12)$$

where $f(\mathbf{x}_{i,j}^{t,\text{old}}; \Theta_{\mathcal{O}}^{t-1}) \in \mathbb{R}^{\mathcal{K}^{t,\text{old}}}$ denotes the logits output for the given input $\mathbf{x}_{i,j}^{t,\text{old}}$ via $\Theta_{\mathcal{O}}^{t-1}$, and $f_{\mathcal{K}^{t,\text{old}}}(\mathbf{x}_{i,j}^{t,\text{old}}; \Theta_{\mathcal{E}_i}^{t,r}) \in \mathbb{R}^{\mathcal{K}^{t,\text{old}}}$ denotes the first $\mathcal{K}^{t,\text{old}}$ values of the logits output for the given input $\mathbf{x}_{i,j}^{t,\text{old}}$ via $\Theta_{\mathcal{E}_i}^{t,r}$. These values represent the logits output for the $\mathcal{K}^{t,\text{old}}$ old classes during the t th incremental task. $\text{sg}(\cdot)$ is a stop-gradient (sg) operation, which is used to prevent

Algorithm 2: Local Training for the Edge Server $\mathcal{E}_i$

Input: current incremental task id t ; received global model $\Theta_{\mathcal{O}}^{t,r}$; number of local epochs E ; learning rate η ;

Output: updated local model $\Theta_{\mathcal{E}_i}^{t,r}$

```

1 Edge  $\mathcal{E}_i$  executes LocalTraining( $t, \mathcal{E}_i, \Theta_{\mathcal{O}}^{t,r}$ ):
2  $\Theta_{\mathcal{E}_i}^{t,r} \leftarrow \Theta_{\mathcal{O}}^{t,r}$ 
3 for epoch  $e = 1, 2, \dots, E$  do
4   for each batch  $\{\mathcal{X}_{i,b}^t, \mathcal{Y}_{i,b}^t\}$  of  $\mathcal{D}_i^t \cup \mathcal{M}_i$  do
5     Calculate  $\mathcal{L}_{CE}$  via (9)
6      $\mathcal{L} \leftarrow \mathcal{L}_{CE}$ 
7     if  $t > 1$  then
8        $\triangleright$  for iCaRL:
9          $\lambda_1 = \lambda_2 = 1$ 
10        Calculate  $\mathcal{L}$  via (13)
11        $\triangleright$  for WA:
12          $\lambda_1 = \frac{\mathcal{K}^t}{\mathcal{K}^{t,old} + \mathcal{K}^t}, \lambda_2 = \frac{\mathcal{K}^{t,old}}{\mathcal{K}^{t,old} + \mathcal{K}^t}$ 
13        Calculate  $\mathcal{L}$  via (13)
14        $\triangleright$  for DER:
15        Calculate auxiliary loss  $\mathcal{L}'_{CE}$  via (9)
16         $\mathcal{L} \leftarrow \mathcal{L} + \mathcal{L}'_{CE}$ 
17      $\Theta_{\mathcal{E}_i}^{t,r} \leftarrow \Theta_{\mathcal{E}_i}^{t,r} - \eta \nabla \mathcal{L}$ 
18 return  $\Theta_{\mathcal{E}_i}^{t,r}$  to the cloud  $\mathcal{O}$ 

```

gradients from being propagated through a specific part of a computational graph. $\Theta_{\mathcal{O}}^{t-1}$ represents the best model stored in the cloud $\mathcal{O}$ during the $(t-1)$ th incremental task. τ is the temperature scalar, with a default setting of 2.

The overall loss combines the classification loss and the distillation loss as follows:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{CE} + \lambda_2 \mathcal{L}_{KD} \quad (13)$$

where λ_1 and λ_2 are used to denote the importance of learning new classes and maintaining old classes, respectively. The detailed procedure for local training is shown in Algorithm 2.

2) *Global Aggregation*: In the t th incremental task, for each communication round r , the cloud $\mathcal{O}$ aggregates all locally updated models $\{\Theta_{\mathcal{E}_i}^{t,r}\}_{i=1}^n$ uploaded by edge servers $\mathcal{E}$ and computes the global model $\Theta_{\mathcal{O}}^{t,r+1}$ for the next communication round $r+1$ as follows:

$$\Theta_{\mathcal{O}}^{t,r+1} = \sum_{i=1}^n \frac{|\mathcal{D}_i^t \cup \mathcal{M}_i| \Theta_{\mathcal{E}_i}^{t,r}}{\sum_{j=1}^n |\mathcal{D}_j^t \cup \mathcal{M}_j|}. \quad (14)$$

As the aggregation of the global model progresses at the cloud $\mathcal{O}$ over the communication round r : $1 \rightarrow R$, within the current t th incremental task, we compare the performance of the models and obtain the best global model to store as

$$\Theta_{\mathcal{O}}^{t,r} = \arg \max_{\Theta_{\mathcal{O}}^{t,r}} \{ \text{eval}(\Theta_{\mathcal{O}}^{t,r}) \mid r \bmod e = 0, 1 \leq r \leq R \} \quad (15)$$

where $\text{eval}(\cdot)$ is the evaluation function in terms of the accuracy metric and $e \in \mathbb{Z}_{\leq R}^+$ is the evaluation interval.

D. Complexity Analysis

According to Algorithm 1, the overall computational complexity of HFIN for each incremental task can be decomposed into the following components.

- 1) *WTO Algorithmic Complexity*: From (8), the WTO algorithm primarily employs a priority-based sorting mechanism, specifically a bubble sort. Its computational complexity is given by $O(N \log N)$, where N denotes the number of client devices per edge server.
- 2) *Model Training Complexity*: The training complexity for the current incremental task is expressed as $O(nRX)$, where X represents the training complexity per epoch for a 1-D-CNN model.
- 3) *Rehearsal Algorithm Complexity*: Following [25], considering the sorting algorithm implemented within the rehearsal algorithm, which utilizes bubble sort. Its complexity is determined as $O(nMD \log D)$, where M denotes the size of the exemplar memory $\mathcal{M}_i$ and D represents the size of the data set $\mathcal{D}_i$.

Consequently, the comprehensive computational complexity of HFIN can be succinctly formulated as follows:

$$O(nT(N \log N + RX + MD \log D)) \quad (16)$$

where n is the number of edge servers, T is the number of tasks, and R is the number of communication rounds.

V. EVALUATION METRICS

To assess the capacity of NID, we employ widely used metrics, such as F1-score and accuracy in the context of a multiclass classification task. Additionally, we leverage the macro-average to compute average values in the context of the multiclass NID.

A. F1-score

This metric represents the harmonic average of precision and recall, formulated as follows:

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot \text{TP}}{2 \cdot \text{TP} + \text{FN} + \text{FP}}. \quad (17)$$

This metric offers a balanced accuracy measure for multiclass classification models, particularly applicable in tasks involving imbalanced data sets, such as the NetFlow traffic attack classification. The macro-F1 for the t th incremental task is calculated as follows:

$$\begin{aligned} \text{macro-F1} &= \frac{1}{\mathcal{K}^{t,old} + \mathcal{K}^t} \sum_{k=1}^{\mathcal{K}^{t,old} + \mathcal{K}^t} F1_k \\ &= \frac{1}{\mathcal{K}^{t,old} + \mathcal{K}^t} \sum_{k=1}^{\mathcal{K}^{t,old} + \mathcal{K}^t} \frac{2 \cdot \text{TP}_k}{2 \cdot \text{TP}_k + \text{FN}_k + \text{FP}_k} \end{aligned} \quad (18)$$

where $F1_k$ denotes the F1-score for the k th class; and TP_k , FN_k , and FP_k represent the true positive, false negative, and false positive values for the k th class in the confusion matrix.

B. Accuracy

The multiclass accuracy for the t th incremental task is given by

$$\text{Accuracy} = \frac{\sum_{k=1}^{\mathcal{K}^{t,\text{old}} + \mathcal{K}^t} \text{TP}_k}{\sum_{k=1}^{\mathcal{K}^{t,\text{old}} + \mathcal{K}^t} (\text{TP}_k + \text{FN}_k)} \quad (19)$$

where TP_k and FN_k denote the true positive and false negative values for the k th class in the confusion matrix, respectively.

VI. EXPERIMENT AND RESULTS

In this section, we first introduce the experimental settings, including the data sets, experimental setup, and benchmarks. Then we compare the proposed HFIN with other baselines in terms of accuracy and F1-score.

A. Data Sets

1) *NF-ToN-IoT-v2* [36]: An IoT network data set, rooted in NetFlow, involving diverse attack categories. Its NetFlow records are generated using publicly available PCAP files from the ToN-IoT data set. The data set comprises a total of 16 940 496 data records, with 6 099 469 (36.01%) flows of *Benign* and 10 841 027 (63.99%) flows of attacks. Specifically, these attacks consist of nine distinct types: 1) *Scanning*; 2) *XSS*; 3) *DDoS*; 4) *Password*; 5) *DoS*; 6) *Injection*; 7) *Backdoor*; 8) *MITM*; and 9) *Ransomware*. Besides, each data flow is made up of 45 NetFlow features.

2) *NF-UQ-NIDS-v2* [36]: A comprehensive data set, integrated from four data sets: 1) NF-UNSW-NB15-v2; 2) NF-ToN-IoT-v2; 3) NF-BoT-IoT-v2; and 4) NF-CSE-CIC-IDS2018-v2. This unified data set comprises data flows from diverse network configurations and various attack scenarios. Notably, the attack categories have been restructured to merge into parent categories. This data set comprises a total of 75 987 976 records, with 25 165 295 (33.12%) flows of *Benign* and 50 822 681 (66.88%) flows of attacks. Specifically, these attacks consist of 20 distinct types: 1) *DDoS*; 2) *DoS*; 3) *Scanning*; 4) *Reconnaissance*; 5) *XSS*; 6) *Password*; 7) *Injection*; 8) *Bot*; 9) *Brute Force*; 10) *Infiltration*; 11) *Exploits*; 12) *Fuzzers*; 13) *Backdoor*; 14) *Generic*; 15) *MITM*; 16) *Ransomware*; 17) *Theft*; 18) *Analysis*; 19) *Shellcode*; and 20) *Worms*. Similarly, each data flow is made up of 46 NetFlow features.

B. Experimental Setup

In the experimental setup phase, we initiate preprocessing steps for the data sets. Due to the highly imbalanced interclass distribution in both NF-ToN-IoT-v2 and NF-UQ-NIDS-v2, we perform downsampling on both data sets. Specifically, for NF-ToN-IoT-v2, we randomly sample half of the data instances from categories with class IDs ranging from 1 to 7. For NF-UQ-NIDS-v2, we randomly sample 1/8 of the data instances from categories with class IDs ranging from 1 to 3 and 1/3 of the data for class IDs ranging from 4 to 8. Subsequently, for NF-ToN-IoT-v2, we remove four dimensions (i.e., “IPV4_SRC_ADDR,” “IPV4_DST_ADDR,” “Label,” and “Attack”) from the original 45 NetFlow features.

TABLE II
DETAILED DIVISION OF INCREMENTAL TASKS FOR iNF-ToN-IoT-v2

iNF-ToN-IoT-v2				
Class ID	Class	Incremental Task		
		$T = 5$	$T = 2$	
1	Benign	$t = 1$	$t = 1$	
2	Scanning			
3	XSS			
4	DDoS	$t = 2$		
5	Password	$t = 3$		
6	DoS			
7	Injection	$t = 4$	$t = 2$ (Last)	
8	Backdoor			
9	MITM	$t = 5$ (Last)		
10	Ransomware			

For NF-UQ-NIDS-v2, in addition to the aforementioned four dimensions, we further excluded the “Data set” dimension from the original 46 NetFlow features. Following this, we perform label encoding on these two data sets and remove redundant rows. For both NF-ToN-IoT-v2 and NF-UQ-NIDS-v2, 40% of the processed data is randomly selected as the testing set, with the remaining portion as the training set. Finally, we apply data normalization to these data sets, respectively.

C. Benchmark Protocol

For the sake of fair comparison with the benchmark CIL methods in the HFIN setting, we follow the benchmark protocol for CIL in [25]. However, the aforementioned benchmark primarily focuses on image classification tasks, which significantly differ from the NID task. To better simulate and experimentally explore this process, we establish the following instantiations of the above protocol.

1) *iNF-ToN-IoT-v2 Benchmark*: We use the NF-ToN-IoT-v2 data and train all 10 classes in batches of 2 or 5 new classes at a time, corresponding to a total of 5 and 2 incremental tasks (i.e., $T = 5, 2$), respectively. The detailed division of each incremental task is depicted in Table II.

2) *iNF-UQ-NIDS-v2 Benchmark*: We use the NF-UQ-NIDS-v2 data and train all 21 classes in batches of 2, 4, or 10 new classes at a time, corresponding to a total of 10, 5, and 2 incremental tasks (i.e., $T = 10, 5, 2$), respectively. The detailed division for each incremental task is provided in Table III.

Additionally, we set the label space distribution parameter α for Benign to 0.3 (i.e., $\alpha_1 = 0.3$) and for all attack classes to 0.8 (i.e., $\alpha_k = 0.8$ where $1 < k \leq K$).

D. Implementation Details

We employ Pytorch to conduct our experiment, utilizing the SGD optimizer with a momentum of 0.9. For the first incremental task ($t = 1$), the initial learning rate is set to $1e^{-2}$, and the weight decay is $5e^{-4}$. For the following incremental tasks ($t > 1$), the initial learning rate is set to $2e^{-2}$, and the weight decay is set to $2e^{-4}$. Additionally, we schedule the learning rate using the multistep strategy, with a decay factor

TABLE III
DETAILED DIVISION OF INCREMENTAL TASKS FOR iNF-UQ-NIDS-v2

iNF-UQ-NIDS-v2				
Class ID	Class	Incremental Task		
		$T = 10$	$T = 5$	$T = 2$
1	Benign	$t = 1$	$t = 1$	$t = 1$
2	DDoS			
3	DoS			
4	Scanning	$t = 2$	$t = 2$	
5	Reconnaissance			
6	XSS	$t = 3$		
7	Password			
8	Injection	$t = 4$		
9	Bot			
10	Brute Force	$t = 5$	$t = 3$	
11	Infiltration			
12	Exploits	$t = 6$		
13	Fuzzers			
14	Backdoor	$t = 7$	$t = 4$	
15	Generic			
16	MITM	$t = 8$		
17	Ransomware			
18	Theft	$t = 9$	$t = 5$ (Last)	
19	Analysis			
20	Shellcode	$t = 10$ (Last)		
21	Worms			

of 0.1. For the first incremental task, the milestones are set at the 20th and 30th communication rounds. For the following incremental tasks, the milestones are set at the 40th and 60th rounds. We set communication rounds (R) to 40 for the first incremental task and 80 for the subsequent ones. The batch size is set to 1024, the evaluation interval (e) is set to 5, and the local epoch (E) is set to 1. Throughout the training phase, we adopt automatic mixed precision (AMP) to all methods. For each edge server $\mathcal{E}_i$ under various configurations, the size of the exemplar memory $\mathcal{M}_i$ is fixed set to 500. We set $n = 3$ and $m = 20$ for the hierarchical FL (HFL) setting.

In HFIN, we employ four CIL baselines as follows.

1) *Replay*: Baseline method with exemplar replay.

2) *iCaRL* [25]: Standard CIL baseline, which introduces a herding-based prioritized selection mechanism to filter and retain old exemplars within the exemplar memory. Furthermore, it proposes a representation learning method based on knowledge distillation.

3) *WA* [26]: An enhanced baseline for CIL, which proposes weight aligning that corrects the biased weights in the FC layer after the normal training process to further maintain the fairness between old classes and new classes.

4) *DER* [27]: One of the state-of-the-art methods for CIL, which utilizes a dynamically expandable representation for more effective incremental concept modeling. This is achieved by freezing the previously learned representation and augmenting it with additional feature dimensions from a new learnable feature extractor.

The three methods, iCaRL, WA, and DER, share the same hyperparameter: the temperature scalar τ , which is set to 2.

E. Quantitative Results Analysis

In the cloud-edge-client hierarchical architecture, the fundamental baseline of HFIN is established by integrating the four aforementioned CIL methods with HFL, denoted as “XXX+HFL” (where XXX represents the CIL methods). These baselines adhere to the fundamental principle of maximizing the transmission of NetFlow data for new classes during edge-client data transmission. Our proposed approach, employing a WTO strategy, comprehensively considers the existing model accuracy and the quantity of data collected for each class. This optimization enhances the edge-client data transmission process, consequently further improving the performance of HFIN. The parameter β of WTO^β is introduced to control the smoothness of the transmission weights concerning data quantity during this process. We compare our method with several baselines on iNF-UQ-NIDS-v2 and iNF-ToN-IoT-v2.

1) *Evaluation on iNF-UQ-NIDS-v2*: We conduct three experiments on this benchmark, corresponding to the total incremental tasks (T) of 10, 5, and 2, respectively. The evaluation results for Accuracy metrics are presented in Tables IV–VI, and macro-F1 metrics are illustrated in Figs. 5–7. In Tables IV–VI, we report the performance throughout the entire incremental task process, the average results across all incremental tasks (Avg.), and the improvement of our method compared to the baseline (Δ). From Tables IV–VI, comprehensively assessing the performance of the last incremental task and the average performance, DER consistently achieved the best results among all four class-incremental methods. Although the performance improvement of DER decreases with a reduction in the total number of incremental tasks (an increase in the number of new classes in a single incremental task), DER still demonstrates relatively superior performance. From Tables IV and V, it is evident that as the incremental process advances (with increasing t), the accuracy of iNF-UQ-NIDS-v2 with $T = 10/5$ does not consistently decline. Instead, it initially decreases and then rebounds. This behavior stems from the Accuracy metric, which is susceptible to the influence of categories with higher proportions. As the incremental process moves forward, there is a tendency to prioritize transferring novel categories and those with fewer data instances. However, in later stages, all classes of data can be transmitted within bandwidth constraints. This leads to an initial accuracy decline followed by a rebound.

As shown in Figs. 5–7, the overall trend indicates a decline in macro-F1 performance with the progression of the incremental learning process (as t increases), where DER also maintains a relatively advantageous performance. Combining Accuracy and macro-F1 metrics, it is evident that our proposed WTO shows significant improvement over various hierarchical federated CIL baselines. Regarding hyperparameter settings, performance with $\text{WTO}^{\beta=0.5}$ consistently outperforms $\text{WTO}^{\beta=0}$ (not considering the quantity of data for each class), emphasizing the importance of considering the quantity of data for each class when optimizing edge-client data transmission. Moreover, in the case of the total incremental tasks of 2, the performance with $\text{WTO}^{\beta=1}$ (considering the

TABLE IV
QUANTITATIVE RESULTS ON iNF-UQ-NIDS-v2 FOR INCREMENTAL TASKS WITH $T = 10$ IN TERMS OF ACCURACY METRIC

Method	iNF-UQ-NIDS-v2 ($T = 10$)											Avg.	Δ
	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$	$t = 6$	$t = 7$	$t = 8$	$t = 9$	$t = 10$			
Replay+HFL	99.1	76.6	81.1	71.7	60.5	69.5	80.8	75.7	81.2	83.9	78.0	—	
Ours (Replay+HFL+WTO $^{\beta=0}$)	99.1	96.2	88.6	87.4	72.1	73.1	81.3	78.3	81.3	85.6	84.3	$\uparrow 6.3$	
Ours (Replay+HFL+WTO $^{\beta=0.5}$)	99.0	96.4	93.7	85.9	74.3	71.0	81.5	79.9	82.5	84.5	84.9	$\uparrow 6.8$	
Ours (Replay+HFL+WTO $^{\beta=1}$)	99.2	96.0	94.3	89.0	74.1	72.9	77.6	75.8	78.8	80.5	83.8	$\uparrow 5.8$	
iCaRL+HFL	99.1	76.5	80.8	70.1	54.9	66.4	76.4	73.5	79.9	82.9	76.0	—	
Ours (iCaRL+HFL+WTO $^{\beta=0}$)	99.1	96.2	92.5	87.2	64.7	67.9	79.0	77.2	80.9	84.1	82.9	$\uparrow 6.8$	
Ours (iCaRL+HFL+WTO $^{\beta=0.5}$)	99.0	96.4	94.4	86.3	74.0	72.7	83.7	81.2	82.4	84.3	85.4	$\uparrow 9.4$	
Ours (iCaRL+HFL+WTO $^{\beta=1}$)	99.2	96.2	94.4	89.2	75.9	75.5	80.5	77.7	82.6	84.4	85.6	$\uparrow 9.5$	
WA+HFL	99.1	76.0	81.5	66.1	47.6	56.4	60.1	66.9	66.7	72.4	69.3	—	
Ours (WA+HFL+WTO $^{\beta=0}$)	99.1	96.0	92.1	88.5	72.8	73.1	78.6	78.9	76.2	76.3	83.2	$\uparrow 13.9$	
Ours (WA+HFL+WTO $^{\beta=0.5}$)	99.0	96.1	94.0	86.8	74.6	73.6	79.0	77.9	78.8	81.4	84.1	$\uparrow 14.8$	
Ours (WA+HFL+WTO $^{\beta=1}$)	99.2	96.1	93.5	90.9	75.1	74.0	81.0	75.8	77.2	79.6	84.2	$\uparrow 15.0$	
DER+HFL	99.3	79.9	80.9	73.1	61.0	73.0	83.9	81.4	84.8	86.5	80.4	—	
Ours (DER+HFL+WTO $^{\beta=0}$)	99.3	96.1	91.2	90.7	84.2	76.2	83.6	84.7	88.8	89.4	88.4	$\uparrow 8.0$	
Ours (DER+HFL+WTO $^{\beta=0.5}$)	99.3	96.4	94.0	91.9	75.5	75.9	86.0	83.7	87.0	90.3	88.0	$\uparrow 7.6$	
Ours (DER+HFL+WTO $^{\beta=1}$)	99.0	96.2	94.1	90.4	81.3	73.9	86.2	82.9	87.2	88.2	87.9	$\uparrow 7.5$	

TABLE V
QUANTITATIVE RESULTS ON iNF-UQ-NIDS-v2 AND iNF-ToN-IoT-v2 FOR INCREMENTAL TASKS WITH $T = 5$ IN TERMS OF ACCURACY METRIC

Method	iNF-UQ-NIDS-v2 ($T = 5$)							iNF-ToN-IoT-v2 ($T = 5$)						
	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$	Avg.	Δ	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$	Avg.	Δ
Replay+HFL	96.1	78.4	54.5	70.3	27.4	65.3	—	98.0	90.8	73.5	52.7	60.2	75.1	—
Ours (Replay+HFL+WTO $^{\beta=0}$)	96.1	88.5	72.4	78.9	77.2	82.6	↑17.3	98.0	97.7	90.4	72.6	63.5	84.5	↑9.4
Ours (Replay+HFL+WTO $^{\beta=0.5}$)	96.3	92.3	72.8	80.8	78.4	84.1	↑18.8	97.6	97.8	93.3	79.1	66.6	86.9	↑11.8
Ours (Replay+HFL+WTO $^{\beta=1}$)	96.1	92.0	72.4	80.2	80.9	84.3	↑19.0	97.8	98.1	94.2	83.3	67.8	88.2	↑13.2
iCaRL+HFL	96.1	84.0	56.2	74.6	75.1	77.2	—	98.0	94.2	74.5	54.6	57.2	75.7	—
Ours (iCaRL+HFL+WTO $^{\beta=0}$)	96.1	91.5	72.2	78.2	79.1	83.4	↑6.2	98.0	97.9	91.5	72.2	70.9	86.1	↑10.4
Ours (iCaRL+HFL+WTO $^{\beta=0.5}$)	96.3	92.2	74.0	80.8	73.2	83.3	↑6.1	97.6	97.8	93.2	78.7	68.0	87.0	↑11.4
Ours (iCaRL+HFL+WTO $^{\beta=1}$)	96.1	91.9	75.2	80.0	81.6	85.0	↑7.8	97.8	98.0	79.4	80.9	64.5	84.1	↑8.4
WA+HFL	96.1	78.2	56.6	65.5	75.5	74.4	—	98.0	93.1	75.7	60.6	58.6	77.2	—
Ours (WA+HFL+WTO $^{\beta=0}$)	96.1	91.6	73.5	79.0	78.0	83.6	↑9.3	98.0	97.9	90.0	76.2	66.1	85.6	↑8.4
Ours (WA+HFL+WTO $^{\beta=0.5}$)	96.3	92.4	76.3	81.2	78.7	85.0	↑10.6	97.6	97.8	93.7	82.3	73.4	89.0	↑11.8
Ours (WA+HFL+WTO $^{\beta=1}$)	96.1	92.0	74.9	80.1	79.9	84.6	↑10.3	97.8	98.0	96.0	82.2	71.1	89.0	↑11.8
DER+HFL	96.3	79.9	68.6	83.0	86.2	82.8	—	97.9	94.2	77.4	61.1	66.4	79.4	—
Ours (DER+HFL+WTO $^{\beta=0}$)	96.3	89.4	73.6	82.9	86.6	85.8	↑3.0	97.9	97.9	92.9	76.8	71.0	87.3	↑7.9
Ours (DER+HFL+WTO $^{\beta=0.5}$)	96.2	91.9	75.8	83.1	87.4	86.9	↑4.1	97.8	98.0	95.5	78.9	72.3	88.5	↑9.1
Ours (DER+HFL+WTO $^{\beta=1}$)	96.1	92.1	77.5	82.1	85.0	86.6	↑3.8	97.8	98.1	95.8	82.6	68.9	88.7	↑9.3

quantity of data for each class without smoothing) is significantly worse than WTO $^{\beta=0.5}$ and WTO $^{\beta=0}$, even inferior to the baseline. This further underscores the importance of smoothing the quantity of data for each class when optimizing edge-client data transmission. This is because directly calculating weights based on the quantity of data for each category can result in substantial differentiation in transmission weights, especially in the case of highly imbalanced data quantities for multiple classes in NID data sets. This differentiation makes it challenging to comprehensively and adequately consider data transmission for all classes.

2) *Evaluation on iNF-ToN-IoT-v2*: We conduct two experiments on this benchmark, corresponding to the total incremental tasks (T) of 5 and 2, respectively. The evaluation results for Accuracy metrics are presented in Tables V and VI, and macro-F1 metrics are illustrated in Figs. 8 and 9. It can be seen from Tables V and VI that DER consistently achieved the best results among all

four class-incremental methods. Observing Figs. 8 and 9, a general trend of performance decline is observed with increasing t . Additionally, our proposed WTO shows significant improvement over various hierarchical federated CIL baselines. Considering hyperparameter settings and comprehensively assessing the performance for the last incremental task (Last) and the Avg., performance with WTO $^{\beta=0.5}$ consistently outperforms WTO $^{\beta=0}$, confirming the importance of considering the quantity of data for each class when optimizing edge-client data transmission. The performance of WTO $^{\beta=0.5}$ also surpasses WTO $^{\beta=1}$, further validating the importance of smoothing the quantity of data for each class when optimizing edge-client data transmission.

In summary, our proposed WTO demonstrates a significant improvement over all four federated CIL baselines, highlighting its compatibility and applicability. Notably, the setting with WTO $^{\beta=0.5}$ (considering the quantity of data for each class with smoothing) achieves the overall optimal

TABLE VI
QUANTITATIVE RESULTS ON iNF-UQ-NIDS-v2 AND iNF-ToN-IoT-v2 FOR INCREMENTAL TASKS WITH $T = 2$ IN TERMS OF ACCURACY METRIC

Method	iNF-UQ-NIDS-v2 ($T = 2$)				iNF-ToN-IoT-v2 ($T = 2$)			
	$t = 1$	$t = 2$	Avg.	Δ	$t = 1$	$t = 2$	Avg.	Δ
Replay+HFL	92.3	67.2	79.7	—	96.3	58.0	77.2	—
Ours (Replay+HFL+WTO $^{\beta=0}$)	92.3	70.7	81.5	$\uparrow 1.8$	96.3	83.1	89.7	$\uparrow 12.6$
Ours (Replay+HFL+WTO $^{\beta=0.5}$)	93.3	74.6	83.9	$\uparrow 4.2$	96.6	85.7	91.2	$\uparrow 14.0$
Ours (Replay+HFL+WTO $^{\beta=1}$)	29.5	40.7	35.1	$\downarrow 45.3$	92.5	89.9	91.2	$\uparrow 14.1$
iCaRL+HFL	92.3	70.8	81.6	—	96.3	58.4	77.4	—
Ours (iCaRL+HFL+WTO $^{\beta=0}$)	92.3	77.0	84.6	$\uparrow 3.1$	96.3	75.6	86.0	$\uparrow 8.6$
Ours (iCaRL+HFL+WTO $^{\beta=0.5}$)	93.3	74.5	83.9	$\uparrow 2.3$	96.6	88.2	92.4	$\uparrow 15.1$
Ours (iCaRL+HFL+WTO $^{\beta=1}$)	29.5	39.9	34.7	$\downarrow 46.9$	92.5	88.8	90.7	$\uparrow 13.3$
WA+HFL	92.3	76.6	84.5	—	96.3	54.2	75.3	—
Ours (WA+HFL+WTO $^{\beta=0}$)	92.3	79.3	85.8	$\uparrow 1.4$	96.3	78.3	87.3	$\uparrow 12.0$
Ours (WA+HFL+WTO $^{\beta=0.5}$)	93.3	80.3	86.8	$\uparrow 2.3$	96.6	86.5	91.6	$\uparrow 16.3$
Ours (WA+HFL+WTO $^{\beta=1}$)	29.5	41.1	35.3	$\downarrow 46.9$	92.5	89.5	91.0	$\uparrow 15.7$
DER+HFL	92.2	78.8	85.5	—	96.6	61.6	79.1	—
Ours (DER+HFL+WTO $^{\beta=0}$)	92.2	80.4	86.3	$\uparrow 0.8$	96.6	78.1	87.3	$\uparrow 8.2$
Ours (DER+HFL+WTO $^{\beta=0.5}$)	93.0	82.0	87.5	$\uparrow 2.0$	96.7	87.0	91.9	$\uparrow 12.8$
Ours (DER+HFL+WTO $^{\beta=1}$)	29.1	43.3	36.2	$\downarrow 46.8$	94.6	87.5	91.0	$\uparrow 11.9$

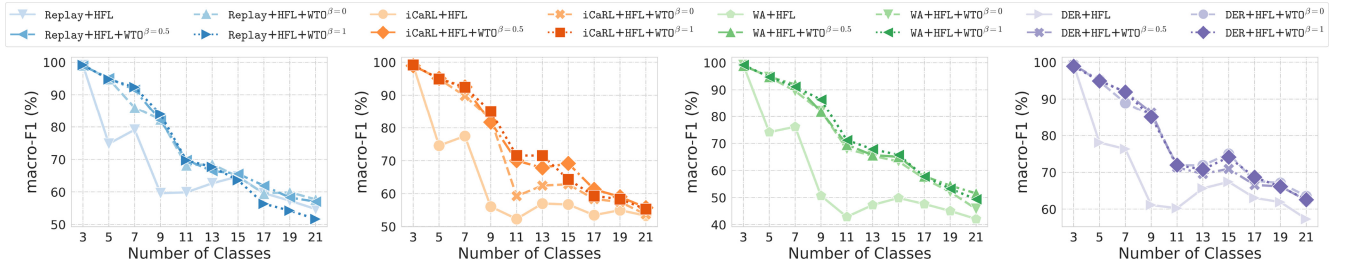


Fig. 5. Qualitative analysis of various methods on iNF-UQ-NIDS-v2 for incremental tasks with $T = 10$ in terms of macro-F1 metric.

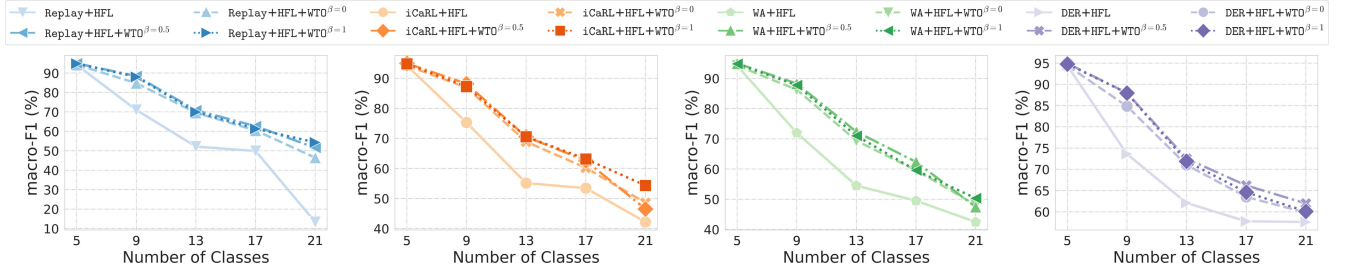


Fig. 6. Qualitative analysis of various methods on iNF-UQ-NIDS-v2 for incremental tasks with $T = 5$ in terms of macro-F1 metric.

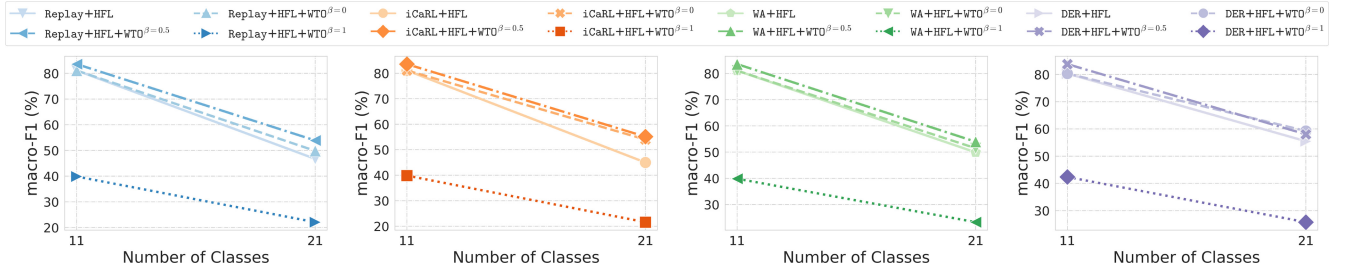


Fig. 7. Qualitative analysis of various methods on iNF-UQ-NIDS-v2 for incremental tasks with $T = 2$ in terms of macro-F1 metric.

results, further illustrating that weighted data transmission (WTO) strategy, by comprehensively considering the accuracy of the current model and smoothing the quantity of data for each class, enhances the overall performance of HFIN.

VII. CONCLUSION

This article innovatively presents the HFIN as a robust solution to the challenges of NID within the rapidly evolving domain of IIoT. Our approach effectively addresses the ever-evolution attack surface and unique vulnerabilities brought

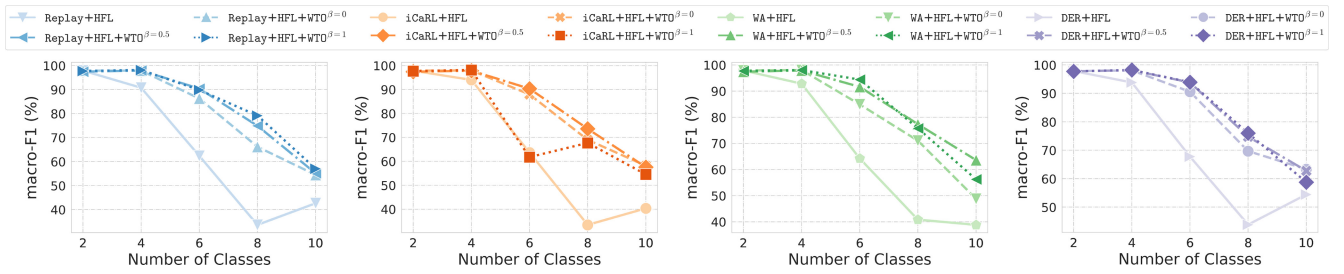


Fig. 8. Qualitative analysis of various methods on iNF-ToN-IoT-v2 for incremental tasks with $T = 5$ in terms of macro-F1 metric.

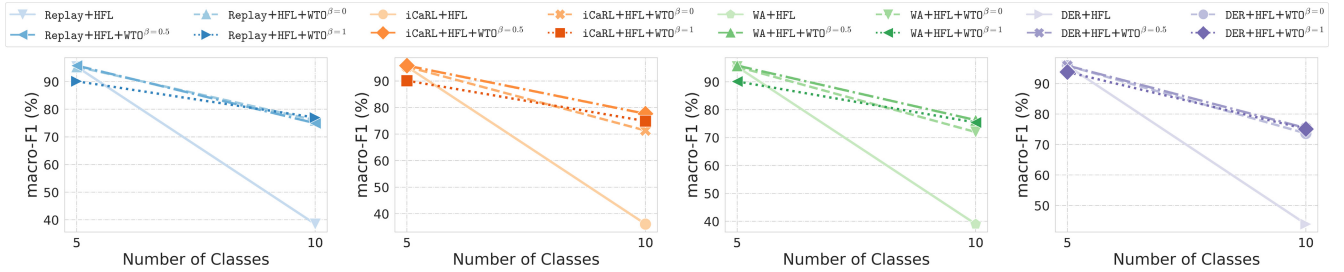


Fig. 9. Qualitative analysis of various methods on iNF-ToN-IoT-v2 for incremental tasks with $T = 2$ in terms of macro-F1 metric.

about by the advent of collaborative manufacturing. By leveraging decentralized model training across a network of IIoT devices, the HFIN approach not only ensures data privacy but also enhances dynamical and continuous detection capabilities through the utilization of distributed data sources. The integration of an edge-client WTO within the HFIN approach represents a significant advancement, effectively reconciling the need for rigorous intrusion detection with the operational constraints of IIoT devices. This balance is crucial for maintaining the comprehensive detection capability of the global model against various network attacks while optimizing resource usage. Empirical evaluations of the HFIN on the NF-UQ-NIDS-v2 and NF-ToN-IoT-v2 data sets confirm its superiority over traditional models, demonstrating an approximate 10% improvement in both accuracy and F1-score. These results validate the effectiveness of the HFIN approach in a real-world context and highlight its potential to significantly enhance the security posture of collaborative manufacturing environments. The success of the HFIN approach paves the way for future advancements in cybersecurity strategies, aligning with the dynamic and interconnected nature of modern industrial ecosystems.

REFERENCES

- [1] K. Lu, Q. Han, G. Zhu, and B. Huang, "Research on the whole process security system framework of network collaborative manufacture," in *Proc. Chin. Control Decis. Conf. (CCDC)*, 2020, pp. 5530–5534.
- [2] J. Wang, C. Xu, J. Zhang, J. Bao, and R. Zhong, "A collaborative architecture of the industrial Internet platform for manufacturing systems," *Robot. Comput. Integr. Manuf.*, vol. 61, Feb. 2020, Art. no. 101854.
- [3] W. Iqbal, H. Abbas, M. Daneshmand, B. Rauf, and Y. A. Bangash, "An in-depth analysis of IoT security requirements, challenges, and their countermeasures via software-defined security," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 10250–10276, Oct. 2020.
- [4] (ATT Telecommun. Co., Dallas, TX, USA). *The CEOS Guide to Data Security Protect Your Data Through Innovation..* (2016). [Online]. Available: <https://www.business.att.com/cybersecurity/docs/vol5-datasecurity.pdf>
- [5] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, "Network intrusion detection for IoT security based on learning techniques," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2671–2701, 3rd Quart., 2019.
- [6] (Palo Alto Netw. Cybersecur. Co., Santa Clara, CA, USA). *2020 Unit 42 IoT Threat Report.* (2020). [Online]. Available: <https://iotbusinessnews.com/download/white-papers/UNIT42-IoT-Threat-Report.pdf>
- [7] J. Lee, J. Kim, and J. Seo, "Cyber attack scenarios on smart city and their ripple effects," in *Proc. Int. Conf. Platf. Technol. Service (PlatCon)*, 2019, pp. 1–5.
- [8] B. Mukherjee, L. T. Heberlein, and K. N. Levitt, "Network intrusion detection," *IEEE Netw.*, vol. 8, no. 3, pp. 26–41, May/Jun. 1994.
- [9] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Comput. Secur.*, vol. 28, nos. 1–2, pp. 18–28, 2009.
- [10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recog.*, 2016, pp. 770–778.
- [11] J. Gu et al., "Recent advances in convolutional neural networks," *Pattern Recognit.*, vol. 77, pp. 354–377, May 2018.
- [12] Y. Yu, X. Si, C. Hu, and J. Zhang, "A review of recurrent neural networks: LSTM cells and network architectures," *Neural Comput.*, vol. 31, no. 7, pp. 1235–1270, Jul. 2019.
- [13] Y. Lin, J. Wang, Y. Tu, L. Chen, and Z. Dou, "Time-related network intrusion detection model: A deep learning method," in *Proc. IEEE Glob. Commun. Conf. (GLOBECOM)*, 2019, pp. 1–6.
- [14] H. He, X. Sun, H. He, G. Zhao, L. He, and J. Ren, "A novel multimodal-sequential approach based on multi-view features for network intrusion detection," *IEEE Access*, vol. 7, pp. 183207–183221, 2019.
- [15] B. Hu et al., "A deep one-class intrusion detection scheme in software-defined industrial networks," *IEEE Trans. Ind. Informat.*, vol. 18, no. 6, pp. 4286–4296, Jun. 2022.
- [16] G. Andresini, A. Appice, and D. Malerba, "Nearest cluster-based intrusion detection through convolutional neural networks," *Knowl. Based Syst.*, vol. 216, Mar. 2021, Art. no. 106798.
- [17] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, nos. 1–2, pp. 1–210, 2021.
- [18] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Stat.*, 2017, pp. 1273–1282.
- [19] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.

- [20] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh, A. Dehghantanha, and G. Srivastava, "Federated-learning-based anomaly detection for IoT security attacks," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2545–2554, Feb. 2022.
- [21] H. Liu et al., "Blockchain and federated learning for collaborative intrusion detection in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 6073–6084, Jun. 2021.
- [22] X. Sun et al., "A hierarchical federated learning-based intrusion detection system for 5G smart grids," *Electronics*, vol. 11, no. 16, p. 2627, 2022.
- [23] P. Ruzafa-Alcázar et al., "Intrusion detection based on privacy-preserving federated learning for the industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1145–1154, Feb. 2023.
- [24] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 12, pp. 2935–2947, Nov. 2017.
- [25] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2001–2010.
- [26] B. Zhao, X. Xiao, G. Gan, B. Zhang, and S.-T. Xia, "Maintaining discrimination and fairness in class incremental learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 13208–13217.
- [27] S. Yan, J. Xie, and X. He, "DER: Dynamically expandable representation for class incremental learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 3014–3023.
- [28] J. Dong et al., "Federated class-incremental learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10164–10173.
- [29] T. Wang, Q. Lv, B. Hu, and D. Sun, "A few-shot class-incremental learning approach for intrusion detection," in *Proc. Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2021, pp. 1–8.
- [30] S. K. Amalapuram, A. Tadwai, R. Vinta, S. S. Channappayya, and B. R. Tamma, "Continual learning for anomaly based network intrusion detection," in *Proc. 14th Int. Conf. Commun. Syst. Netw. (COMSNETS)*, 2022, pp. 497–505.
- [31] F. Cerasuolo, G. Bovenzi, C. Marescalco, F. Cirillo, D. Ciuonzo, and A. Pescape, "Adaptive intrusion detection systems: Class incremental learning for IoT emerging threats," in *Proc. IEEE Int. Conf. Big Data (BigData)*, 2023, pp. 3547–3555.
- [32] L. Du, Z. Gu, Y. Wang, L. Wang, and Y. Jia, "A few-shot class-incremental learning method for network intrusion detection," *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 2, pp. 2389–2401, Apr. 2024.
- [33] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–11.
- [34] K. Zhang, B. Li, X. Tang, D. Wang, and L. Wei, "Path loss measurement and modeling for industrial environment," in *Proc. IEEE 20th Int. Conf. High Perform. Switching Routing (HPSR)*, 2019, pp. 1–5.
- [35] P. S. Bouzinis, P. D. Diamantoulakis, and G. K. Karagiannidis, "Wireless quantized federated learning: A joint computation and communication design," *IEEE Trans. Commun.*, vol. 71, no. 5, pp. 2756–2770, May 2023.
- [36] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Netw. Appl.*, vol. 27, no. 1, pp. 357–370, 2022.



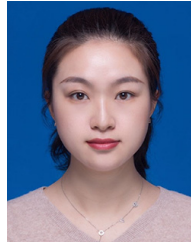
Jingxin Mao received the B.E. degree from Tongji University, Shanghai, China, in 2022, where he is currently pursuing the master's degree in software engineering.

His current research interests include federated learning, semantic segmentation, semi-supervised learning, and incremental learning.



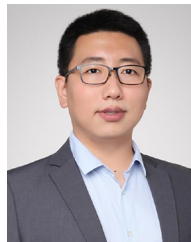
Zhiwei Wei (Graduate Student Member, IEEE) received the master's degree from Tongji University, Shanghai, China, in 2023, where he is currently pursuing the Ph.D. degree with Shanghai Research Institute for Intelligent Autonomous Systems.

His research focuses on vehicular fog computing, industrial service provision, and dynamic resource allocation.



Bing Li (Member, IEEE) received the Ph.D. degree from Tongji University, Shanghai, China, in 2021.

She is currently an Assistant Professor with Tongji University. Her current research interests include UAV communications, wireless resource allocation, and relay communications.



Rongqing Zhang (Member, IEEE) received the B.S. and Ph.D. degrees (with Hons.) from Peking University, Beijing, China, in 2009 and 2014, respectively.

From 2014 to 2018, he was a Research Fellow with Colorado State University, Fort Collins, CO, USA. Since 2019, he has been an Associate Professor with Tongji University, Shanghai, China. He has authored and coauthored three monographs, two book chapters, and over 150 papers in refereed journals and conference proceedings. His current

research interests include vehicular communications and networking, UAV, and connected intelligence.

Dr. Zhang was the co-recipient of the 2017 First Prize in Natural Science, Ministry of Education of China, the 2023 First Prize in Natural Science, Chinese Association of Automation, and the 2023 First Prize in Natural Science, China Institute of Communications. He also received the Best Paper Awards at IEEE ITST'12, ICC'16, GLOBECOM'18, and ICC'19. He is currently serving as the Secretary General for the Connected Intelligence Committee of Chinese Association of Automation, the Vice-Chair for the Information Services Committee of IEEE ComSoc Asian-Pacific Board, and an Associate Editor for IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY and *IET Communications*.



Lingyang Song (Fellow, IEEE) received the Ph.D. degree from the University of York, York, U.K., in 2007.

He was a Research Fellow with the University of Oslo, Oslo, Norway, until rejoining Philips Research U.K. in March 2008. In May 2009, he was with the Department of Electronics, School of Electronics Engineering and Computer Science, Peking University, Beijing, China, and is currently a Boya Distinguished Professor. His main research interests include wireless communication and networks, signal processing, and machine learning.

Dr. Song received the K. M. Stott Prize for Excellent Research from the University of York. He was the recipient of the IEEE Leonard G. Abraham Prize in 2016 and the IEEE Asia-Pacific (AP) Young Researcher Award in 2012. He has been an IEEE Distinguished Lecturer since 2015.